

I. Introduction to Models, Tools, and Agents

- **Limitation of Foundation Models:** Without external functions, even the most advanced foundation model is only a pattern prediction engine, limited to content generation based on its training data. It cannot access new data, interact with external systems, or take actions in its environment.
- **Role of Tools:** Tools enable an AI system to do more, acting as the agent's **"eyes" and "hands"** to perceive and act on the world.
- **Tool Definition:** A tool is a function or program an LLM-based application uses to accomplish a task outside the model's capabilities.
- **Tool Functionality:** Tools broadly fit into two types: they allow a model to **know something** (retrieve data from sources) or to **do something** (perform an action, often by calling an external API).
- **Types of Tools:**
 1. **Function Tools:** External functions defined by the developer that the model can call.
 2. **Built-in Tools:** Tool definitions implicitly provided to the model by the model service (e.g., Google's Gemini API offers Grounding with Google Search, Code Execution, URL Context).
 3. **Agent Tools:** An agent that is invoked as a tool, allowing the primary agent to maintain control over the interaction and process the sub-agent's input/output.
- **Taxonomy of Agent Tools (By Function):** Information Retrieval, Action/Execution, System/API Integration, and Human-in-the-Loop.

II. Best Practices for Tool Design

Effective tool design is essential because the tool documentation is passed directly to the model as part of the request context, instructing the model on how to use it.

Category	Key Practices
Documentation	Use a clear, descriptive, and human-readable name (important for governance/audit logs). Simplify parameter lists and give parameters clear names. Clarify tool descriptions using simple terminology and add targeted examples to refine model behavior.
Functionality	Describe actions, not implementations: Explain <i>what</i> the model needs to do (e.g., "create a bug"), not <i>how</i> to use the tool. Publish tasks, not API

	calls: Tools should encapsulate a specific task the agent performs, not just be thin wrappers over complex, existing APIs. Make tools as granular as possible , limited to a single function.
Input/Output	Design for concise output: Large data responses (tables, files) quickly swamp the LLM's output context and conversation history, adversely affecting performance and cost. Use external systems for large data storage (e.g., temporary database table or Artifact Service). Use validation effectively: Input and output schemas serve as documentation and provide a runtime check on correct tool operation.
Error Handling	Provide descriptive error messages that include instruction to the calling LLM about what to do to address the specific error.

III. Understanding the Model Context Protocol (MCP)

- **Standardization Need:** The ecosystem of tool integrations is fragmented, leading to the "**N x M**" **integration problem**, where custom connections grow exponentially with each new model (N) or tool (M).
- **MCP Introduction:** Anthropic introduced the Model Context Protocol (MCP) in 2024 as an **open standard** to streamline integration, aiming to replace custom integrations with a unified, plug-and-play protocol.
- **Architectural Components (Client-Server Model):**
 - **MCP Host:** Manages the user experience, orchestrates tools, and enforces security policies.
 - **MCP Client:** Maintains connection with the Server, issues commands, and manages the session lifecycle.
 - **MCP Server:** Advertises available tools (tool discovery), executes commands, and formats and returns results; often acts as an adapter or proxy for external tools/APIs.
- **Communication Layer:**
 - **Base Protocol:** MCP uses **JSON-RPC 2.0**.
 - **Transport Mechanisms:** Supports **stdio** (for local environments where the server runs as a subprocess) and **Streamable HTTP** (recommended for remote client-server communication).

- **Key Primitives/Capabilities (Entity Types):**

- **Server-Side Capabilities: Tools** (most broadly supported, core driver of value), Resources, and Prompts.

- **Client-Side Capabilities:** Sampling, Elicitation, and Roots (low adoption currently).

MCP Tool Definition Details

- MCP Tool definitions must conform to a JSON schema. Key fields include:

- name: Unique identifier.

- description: Human- (and LLM-) readable description of functionality.

- inputSchema: JSON schema defining expected parameters.

- outputSchema: JSON schema defining the output structure (optional, but should be treated as required).

- **Tool Results:** Can be **structured** (always returned as a JSON object, requiring output schema validation by clients) or **unstructured** (Text, Audio, or Image data).

- **MCP Error Handling:** A Server can return standard **JSON-RPC errors** (for protocol issues like unknown tools) or return execution errors within the tool results by setting the "isError": true parameter.

IV. Strategic Advantages and Challenges

Capabilities and Strategic Advantages

- **Accelerating Development:** Simplifies integration, reducing development time and cost.

- **Reusable Ecosystem:** Fosters a "plug-and-play" ecosystem where tools are shareable assets.

- **Dynamic Tool Discovery:** MCP applications can discover tools at runtime, allowing for greater adaptability and autonomy.

- **Architectural Flexibility:** Standardizing the agent-tool interface decouples the agent's architecture from implementation, allowing organizations to switch LLM providers or backend services without re-architecting the entire integration layer.

- **Foundations for Governance:** The architecture provides hooks for robust governance, allowing security policies and access controls to be embedded in the MCP server.

Critical Risks and Challenges

- **Context Window Bloat:** Loading definitions for every tool from every connected MCP server into the model's context window consumes tokens, increasing cost/latency, and can **degrade reasoning quality** by making it difficult for the model to identify relevant tools.

- *Note:* A potential future mitigation involves dynamic "tool retrieval" (like RAG) to load only the most relevant subset of tools.

- **Stateful Protocol Challenges:** Using stateful connections for remote servers can complicate architecture and hinder horizontal scaling, especially when integrating with predominantly stateless REST APIs.

- **Enterprise Readiness Gaps:** The core protocol currently lacks robust, standardized features for:

- **Authentication and Authorization.**
 - **Identity Management Ambiguity** (unclear whether the action is initiated by the end-user, the agent, or a generic system account).
 - **Native Observability** (logging, tracing, metrics).

V. Security Threats and Mitigations in MCP

The broad applicability of MCP increases the likelihood and potential severity of security issues, particularly unauthorized actions and data exfiltration.

Risk	Description	Mitigations
Dynamic Capability Injection	MCP servers can dynamically change their tool list without notifying the client, potentially allowing agents to unexpectedly inherit dangerous or unapproved capabilities (e.g., a retrieval agent suddenly gaining financial transaction ability).	Implement an explicit allowlist of permitted MCP tools and servers. Require Mandatory Change Notification (set listChanged flag). Use secure API/Agent Gateways (like Apigee) to filter and apply policies to the tool list returned to the client.
Tool Shadowing	Malicious tool definitions can specify triggers that overshadow legitimate tools,	Prevent Naming Collisions (using LLM-based semantic checks). Require Human-in-the-Loop (HIL) confirmation for all high-risk

	tricking the agent into using the rogue tool to intercept or exfiltrate sensitive data.	operations (e.g., file deletion, modification of production data). Restrict Access to unauthorized MCP servers.
Malicious Tool Definitions & Consumed Contents	Tool descriptor fields (documentation, API signature) can manipulate agent planners into rogue actions, and tool return values can lead to data exfiltration.	Input Validation (to sanitize user inputs) and Output Sanitization (to remove malicious content, API tokens, or credit card numbers from tool returns before feeding them back to the model context). Strict allowlist validation for Resources consumed from third-party servers.
Sensitive Information Leaks	Sensitive information stored in the conversation context can be unintentionally (or intentionally) received by MCP tools. The Elicitation capability also poses a risk if strong guardrails are not implemented, as the policy against requesting sensitive information is unenforced.	Tag inputs and outputs as " tainted " or " not tainted " to track and control the flow of sensitive data. Tools carrying sensitive output should be clearly identified using custom annotations.
No Support for Scope Limiting	The protocol supports only coarse-grained client-server authorization, lacking native support for per-tool or per-resource permissions.	Use scoped credentials that are bound to authorized callers and have short expiration periods. Adhere to the principle of least privilege . Keep secrets and credentials out of the agent context by transmitting them to the server through a side channel.
Confused Deputy Problem (Classic Risk)	A privileged entity (the MCP server, or "deputy") is tricked by an unprivileged entity (the attacker, via a prompt injection attack on the AI model) into misusing its authority to perform an unauthorized action (e.g., retrieving sensitive code it	The MCP server, acting as the deputy, must check if the end-user has permission to perform the action, not just if the MCP server itself has broad privileges.

	was not personally authorized to access).	
--	---	--

Analogy:

The relationship between the foundation model and its tools, standardized by the Model Context Protocol (MCP), is like a highly intelligent **CEO (the LLM)** who is brilliant at strategy and language, but physically constrained. The **Tools** are the specialized staff members (e.g., accountant, driver, security guard) who are the only ones capable of interacting with the outside world (the external APIs and data). The **MCP** is the standardized corporate directory and communication manual that allows the CEO to discover, understand, and issue commands to the appropriate staff members efficiently, ensuring the whole operation works together seamlessly. However, just as an unvetted staff member could leak company secrets or use their corporate access to perform malicious actions, an unvalidated MCP tool creates significant **security risks** for the entire enterprise